

Flamingo: On Load Balancing in DAG-based Consensus Protocols

Zhen Ping Khor
University of Pennsylvania
zpkhor@upenn.edu

Mohammad Javad Amiri
Stony Brook University
amiri@cs.stonybrook.edu

Garvit Gupta
Apple
garvitgupta@icloud.com

Boon Thau Loo
University of Pennsylvania
boonloo@seas.upenn.edu

ABSTRACT

Distributed data management systems deployed in untrusted environments rely on Byzantine Fault-Tolerant (BFT) consensus protocols to tolerate malicious failures. DAG-based BFT protocols improve throughput by letting validators disseminate transactions concurrently and by scaling execution across multiple workers. However, imbalances in workload or resource capacity can still degrade performance significantly. This paper presents Flamingo, a load-balancing protocol for certified DAG-based BFT protocols that addresses imbalance at both the ordering and execution layers. At the ordering layer, Flamingo periodically migrates client accounts away from overloaded validators, adapting to skewed submissions and heterogeneous validator capacity while preserving correctness under Byzantine faults, with migrations taking effect only through the committed log. At the execution layer, Flamingo redistributes committed transactions across executor workers using a deterministic, order-preserving scheduler that balances load and minimizes cross-worker data movement, without centralized coordination or costly distributed commit. Built on top of Narwhal and Tusk, our prototype shows that Flamingo recovers throughput and latency under workload skew, validator heterogeneity, and shifting hotspots, adds negligible overhead when the system is balanced, and needs load balancing in both layers, since resolving only one shifts the bottleneck to the other.

1 INTRODUCTION

Byzantine Fault-tolerant (BFT) protocols [9, 28, 39, 62] allow distributed data management systems deployed in untrusted environments to tolerate malicious node behavior. In traditional BFT protocols, a designated leader receives client transactions and broadcasts them with an assigned order to all replicas. This ensures consistency but creates a resource imbalance, since the leader’s storage, CPU, and bandwidth cap system throughput while the remaining replicas sit underutilized. Moreover, since dissemination and consensus are tightly coupled, the leader failure stalls the entire protocol. Rotating the leader [1, 8, 10–12, 14, 26, 30, 38, 40, 60–62] distributes load more evenly across replicas over time [6, 7, 60], but introduces additional overhead due to more frequent view synchronization [62] and still requires every replica to be provisioned for peak leader load, since any replica may assume the leader role yet only one fully uses its capacity at a time. Multi-proposer protocols [4, 29, 44, 45, 55] aim to mitigate this underutilization by running multiple BFT instances concurrently with each replica leading a parallel instance, but because dissemination and consensus remain tightly coupled, a single node’s failure can still affect the entire protocol.

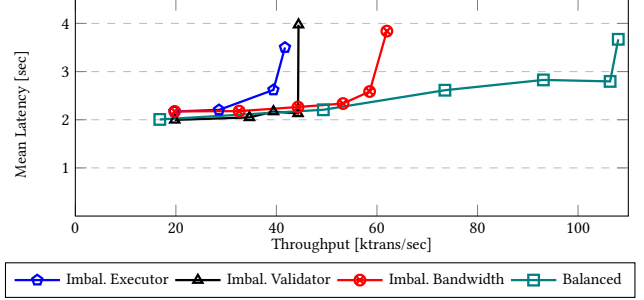


Figure 1: Impact of load imbalance and validator heterogeneity (a single node with limited bandwidth) on performance

DAG-based consensus protocols [3, 13, 16–20, 27, 32, 33, 46, 48, 50, 52–54] address these limitations: validators disseminate transaction blocks concurrently, the blocks form a Directed Acyclic Graph; consensus is decoupled from dissemination, so node failures do not stall transaction processing. Each node consists of an ordering layer, a validator whose workers stream transaction batches in parallel, and an execution layer, whose executor workers each own a disjoint shard of the state; both layers scale horizontally by adding workers.

Despite their scalability, DAG-based protocols are highly sensitive to workload imbalance and validator heterogeneity. Figure 1 demonstrates this using the widely studied DAG-based BFT protocol Narwhal and Tusk [20] in a four-node local deployment. Under balanced resource capacity and load, the protocol achieves approximately 106 ktps. However, imbalances cause significant degradation: reducing one node’s bandwidth to one-third of its peers lowers throughput to ~58 ktps (45% reduction), directing 90% of transactions to a single validator (with the remaining 10% split among three others) yields ~44 ktps (58% reduction), and concentrating 90% of execution on a single executor worker drops throughput to ~39 ktps (63% reduction). These results motivate the need to address load and resource imbalances at both the ordering and execution layers to fully exploit the advantages of DAG-based protocols.

Balancing either layer is complicated by the setting itself. At the ordering layer, clients choose or are pre-assigned to validators, so skewed submissions saturate some validators while others idle, and heterogeneous hardware or fluctuating networks cause the same problem. Having no trusted coordinator makes the re-balancing more challenging: every validator must reach the same migration decision from its own, possibly divergent, DAG view, and a Byzantine validator may misreport its load to trigger harmful load balancing decisions. At the execution layer, skewed access

patterns create hot shards, and distributed transactions spanning multiple executor workers require cross-worker coordination that degrades throughput even at low rates, as locks must be held during coordination [58]. A replicated executor must additionally preserve the consensus-determined order deterministically, even across validators running different numbers of executor workers, which rules out schedulers that reorder batches for locality.

This paper presents *Flamingo*, a load-balancing mechanism for DAG-based BFT protocols addressing imbalance at both ordering and execution layers while running off the critical path and introducing no new trust assumptions.

First, at the ordering layer, Flamingo balances load by changing which validator serves each client. Every client account starts with a validator derived deterministically, and Flamingo periodically migrates accounts from overloaded validators to those with spare capacity, prioritizing receivers with low latency to the client. The load balancer runs on every validator and draws its load signals from certified metadata piggybacked on the DAG, requiring no dedicated metric exchange. Signals are distinguished by how they can be trusted: values that any peer can recompute from certified data determine whether and where load moves, but self-measured values can influence only how much load is migrated, never the safety of the underlying protocol. A migration takes effect only through the committed log, requiring both committed evidence that $2f + 1$ validators reached the same decision and the receiving validator’s committed acceptance. If the assigned validator becomes unresponsive, a bounded relay fallback lets a small deterministic set of validators admit the client’s requests, keeping the account serviceable. We prove three properties of this migration layer: all honest validators agree on every account’s assignment, every correct client’s valid requests eventually commit, and fallback admission is bounded, so the fallback path cannot be abused to inject unbounded extra traffic.

Second, at the execution layer, Flamingo schedules committed transactions across executor workers to balance load while minimizing the data movement required for distributed transactions. Every executor worker receives the full committed batch, which co-located LAN bandwidth readily absorbs, and independently runs the same lightweight deterministic scheduler, producing identical assignments without a central dispatcher or any synchronization of partition state. To perform transactions that access data hosted by multiple workers, Flamingo follows a data fusion model [41]: the assigned worker pulls the remote inputs and performs all writes locally, a single one-way transfer instead of the gather-and-scatter round trip of existing protocols, e.g., PilotFish [37]. It preserves the consensus order at lower scheduling algorithmic complexity compared to Hermes-style prescient routing [42] that reorders batches, which a replicated BFT executor cannot adopt.

We have implemented Flamingo on top of Narwhal and Tusk [20], the foundational certified-DAG design underlying industry-grade systems such as Sui [56]. Since Flamingo does not alter transaction dissemination or the consensus routine, it can be incorporated with other certified DAG-based protocols such as Bullshark [54] and Shoal [53]. Our evaluation shows that Flamingo recovers throughput and latency under submission skew, heterogeneous validator bandwidth, and shifting hotspots. It also scales across validators and executor workers, reduces scheduling overhead compared with

prescient routing [42], and shows that both layers are needed to address their respective bottlenecks. The prototype implements the load measurement and migration decision components of the ordering layer, while the Byzantine safety of the migration-commit lifecycle is established analytically.

Our contributions can be summarized as follows:

- We identify and quantify the sensitivity of DAG-based BFT protocols to load imbalance, showing that submission skew, validator bandwidth heterogeneity, and executor hotspots reduce performance, and that the two affected layers, ordering and execution, both need to be addressed.
- At the ordering layer, we design an account-migration mechanism that derives its signals from certified DAG metadata and activates migrations only through committed evidence and receiver acceptance. We prove assignment agreement, liveness, and bounded fallback admission under Byzantine faults.
- At the execution layer, we design a deterministic, order-preserving, full-batch scheduler that combines load-aware placement with data fusion, reducing each distributed transaction to a single one-way state transfer.
- We implement a prototype of Flamingo and evaluate its performance across various imbalanced conditions, demonstrating throughput and latency recovery under imbalance with negligible overhead when the system is balanced.

2 DAG-BASED BFT CONSENSUS

Traditional BFT protocols funnel both transaction dissemination and ordering through a single leader, capping throughput at the leader’s capacity while other replicas sit underutilized. A recent line of work decouples dissemination from the consensus routine and lets every node broadcast transaction blocks concurrently, organizing them into a Directed Acyclic Graph (DAG) whose edges reference preceding blocks [3, 13, 16–20, 27, 32, 33, 46, 48, 50, 52–54]. Processing a transaction then involves three steps: DAG construction, consensus, and execution. The DAG is built asynchronously to the total order, and each node maintains a local view; the consensus routine ensures a consistent view across nodes where each node derives the total order independently from its local DAG, without any additional communication phase. Since consensus no longer blocks the flow of transactions and every replica builds and processes its DAG in parallel, these protocols utilize the resources of all nodes and achieve substantially higher throughput.

Figure 2 illustrates DAG construction on node n_0 in a 4-node network using the Narwhal and Tusk protocols [20]. In each round, every node packs its client transactions into a block that carries the node’s signature and its causal history, namely certificates of blocks from the preceding round, and broadcasts it. A receiving node validates and stores the block, then returns a signed acknowledgment over the block digest, round number, and creator identity. Once the creator collects $n - f$ distinct acknowledgments, it aggregates them into a Proof of Availability (PoA) certificate and broadcasts it, and all nodes include the certificates they receive in their subsequent blocks. A node advances to the next round after obtaining $n - f$ certificates for the current round, creating the corresponding vertices in its local DAG. For example, n_0 receives certificates from n_0 , n_1 , and n_2 in round 1 and thus creates only these three vertices in

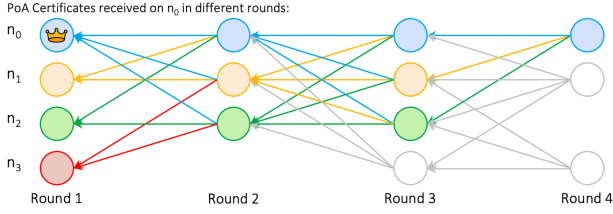


Figure 2: DAG construction on node n_0 . Back edges reflect the PoA certificates that n_0 holds from the preceding round. For example, n_0 receives certificates from n_0 , n_1 , and n_2 in round 1, so round 2 contains only these three vertices. When n_0 is elected as leader, all blocks reachable from the leader block in round 4 are committed.

round 2; the back edges are determined by the certificates stored in the received blocks. Since a PoA certificate carries $n - f$ signatures, at least $f + 1$ honest validators have stored the block, so the block remains available for retrieval when needed for ordering, and quorum intersection prevents equivocation. Because each block references certificates from the previous round, an inductive argument shows that the entire causal history is certified and available, satisfying causality.

In the consensus phase, a node is (randomly) elected as leader for the final round of a wave (e.g., round 4), and each replica independently commits the blocks that have a path to the leader block in its local view, e.g., highlighted blocks in Figure 2, yielding the same total order on all replicas without further communication. Blocks not yet ordered, e.g., gray blocks in Figure 2, are committed in subsequent waves under newly elected leaders. Once the order is established, executor workers execute the transactions. DAG-based protocols typically shard the replicated state among executor workers, each executing the transactions that access its shard.

Despite these gains, our experimental observations in Figure 1 reveal a critical shortcoming: performance is highly sensitive to balanced workloads and homogeneous deployments. When client load is skewed, or nodes differ in capacity, such as bandwidth, throughput and latency degrade sharply. This paper investigates and addresses this problem across the ordering and execution layers of DAG-based BFT protocols.

3 FLAMINGO OVERVIEW

We assume an asynchronous network of $n = 3f + 1$ validators, of which at most f may be Byzantine and exhibit arbitrary, potentially malicious behavior. The adversary is computationally bounded and cannot break cryptographic assumptions. Validators communicate over bidirectional point-to-point channels; a public-key infrastructure (PKI) with digital signatures ensures that messages from honest validators cannot be forged or repudiated, and a collision-resistant hash function $D(\cdot)$ produces fixed-size message digests.

Each node consists of a validator (ordering layer) and an executor (execution layer). The validator comprises a primary and several workers, which receive client transactions, seal and broadcast them as batches, and certify data availability through quorum

acknowledgments. The execution layer partitions state across executor workers, which process committed batches in consensus-determined order.

Figure 3 shows the architecture on n_0 and n_1 . All nodes have the same number of validator workers, since each worker w_i must have a counterpart w_i at every other node to communicate with. The number of executor workers may differ (three for n_0 , two for n_1), and each node may shard state differently. Each validator worker w_i seals transactions from its assigned clients into batches, broadcasts them to its counterparts at the other nodes, and receives the batches those counterparts broadcast. For example, $n_0.w_0$ receives transaction t_1 while $n_1.w_0$ receives t_5 , and each seals its transaction into a batch for the other.

Once a batch is committed, every node receives it, and each transaction is executed by the executor worker whose shard owns the accessed records. For instance, $n_0.w_0$ manages records u_0 to u_{19} and executes transaction t_1 , which accesses records u_{17} and u_{18} . If a transaction, such as t_6 , accesses records across different workers (e.g., u_{02} on $n_0.w_0$ and u_{20} on $n_0.w_1$), Flamingo ensures the relevant state is transferred among the workers.

Flamingo extends DAG-based protocols with two load-balancing mechanisms that run alongside consensus, off its critical path.

Validator-level load balancing (§4). Each client account is assigned to one validator and submits requests to its workers in a round-robin manner. Flamingo periodically rebalances assignments by migrating excess load from overloaded validators to those with spare capacity, using load signals derived from the certified DAG. The algorithm runs on every correct validator, and since it operates on data already carried by the DAG, it requires no dedicated exchange of load reports. A migration takes effect only once two records are committed in sequence: its evidence ($2f + 1$ matching signed decisions), and then the target validator’s acceptance. This committed pair serves as the account’s assignment credential from that point on. Flamingo avoids unnecessary migration when the system is lightly loaded or when no validator has spare capacity to receive load. When the assigned validator is unresponsive, a bounded relay fallback keeps the client account serviceable.

Executor-level load balancing (§5). Within each node, executor workers may be unequally loaded depending on the sharding scheme and workload skewness. Flamingo schedules transactions across executor workers to balance load, while preserving the consensus-determined order so that execution is serializable and every validator reaches the same state from the same committed sequence, even when validators have different numbers of executor workers [58]. Every executor worker receives the full committed batch and independently runs the same lightweight deterministic scheduler, so all workers derive identical assignments without a central dispatcher. Our prototype builds on Narwhal and Tusk [20], whose scale-out validator architecture provides the certified DAG and multi-worker structure described above; §4 specifies the certified-DAG properties Flamingo requires.

4 VALIDATOR LEVEL LOAD BALANCING

Flamingo’s validator-level load balancing migrates accounts across validators to alleviate overload. Load balancing in DAG-based BFT protocols is challenging for two main reasons. First, there is no

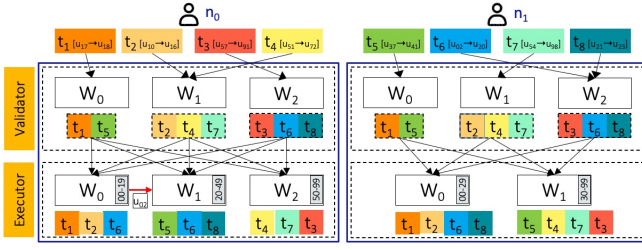


Figure 3: A partial view of Flamingo with nodes n_0 and n_1 , each consisting of 3 validator workers. Node n_0 has 3 executor workers, while n_1 includes 2 executor workers.

central coordinator: all validators must reach the same migration decision while their local DAG views might diverge. Second, a malicious validator can underreport or overreport its load to mislead the load balancer into incorrect migration decisions, so the protocol must be robust to such misreports.

A configuration epoch e fixes the validator committee, public keys, and load-balancing parameters for its duration. Client accounts are authenticated through a Sybil-resistant mechanism, which also issues each account's initial assignment credential, binding the account to a validator derived deterministically from its identifier. We let B_{adv} denote an upper bound on the number of admitted accounts controlled by Byzantine clients in one epoch. We consider a fixed epoch throughout the paper and leave reconfiguration out of scope. The bounded clock-skew assumption introduced below is used only for estimating the magnitude of load-balancing signals and is not required for consensus safety or liveness.

Algorithm 1 runs on each validator every R rounds as *evaluation anchor*, concurrently with consensus and off the critical path. It operates on a stable certified prefix of the DAG: a tracking window of w rounds, from $r-\tau-w+1$ through $r-\tau$, where r is the current round and τ is a configurable value. Excluding the τ most recent rounds, the *unstable frontier*, lets the window stabilize across validator views. The window size demonstrates a freshness-completeness trade-off: a larger τ yields more consistent signals at the cost of staler decisions. Figure 4 shows a window of size w with $\tau = 2$.

Validators run Algorithm 1 over their local certified DAG views (not the committed sequence). A natural alternative is to derive migrations deterministically from the committed DAG, which gives all validators identical inputs but delays rebalancing until consensus orders the relevant load history. Certification happens earlier, so using certified metadata keeps load balancing timely. The downside is that honest validators may temporarily hold different certificate sets and thus reach different migration decisions. Flamingo tolerates this divergence by requiring $2f+1$ matching decisions to form migration evidence, so disagreement simply yields no migration rather than an inconsistent assignment. Since only committed migration evidence and receiver acceptance can change an assignment, divergence in the DAG views can delay rebalancing but never compromises assignment safety.

Although we implement Flamingo on top of Narwhal and Tusk, the design extends to any DAG-based protocols that preserve the certified abstraction, including Bullshark [54] and Shoal [53]. It does not directly carry over to uncertified-DAG protocols such

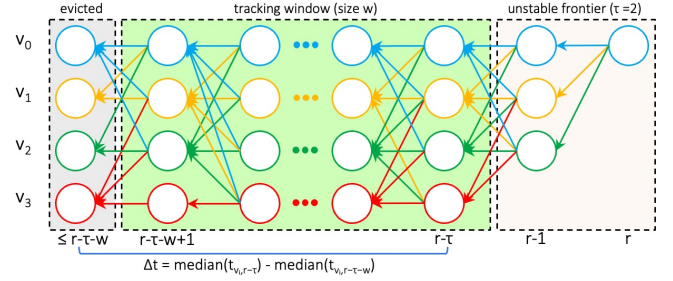


Figure 4: Local DAG view at v_0 . The active tracking window contains w rounds, from $r-\tau-w+1$ through the stable round $r-\tau$ (here $\tau=2$); certificates in the unstable frontier are excluded. Each certificate $c_{v_0,r}$ carries the metadata used by Algorithm 1: block proposal timestamp $t_{v_0,r}$, average batch queue delay $\bar{W}(v_0, r)$, and per-account validated transaction counts $n(v_0, r, a)$ for each client account a . The evaluation interval $\Delta t = \text{median}(t_{v_1, r-\tau}) - \text{median}(t_{v_1, r-w-\tau})$ anchors the throughput and queue-delay estimates.

Algorithm 1 Validator-level load balancing in Flamingo

Input: (1) per-validator DAG-derived throughput τ_v , (2) per-account validated transaction counts $n_{v,a}$, (3) average batch queue delay $\bar{W}_v$, (4) queue-delay baseline $\bar{W}_v$, (5) evaluation interval Δt , (6) anchor of the account's latest effective migration r'_a , current round r , (7) declared capacity C_v , (8) account-validator latency $L_{v,a}$ (latest committed record), (9) donor-qualification threshold $W_{\min}$, (10) migration-cap multiplier α , (11) evaluation interval in rounds R

Output: Set of migration decisions $\mathcal{M} = \{(a, v_{\text{old}}, v_{\text{new}}, r)\}$

- ▷ Identify overloaded donors and underloaded receivers
- 1: $E_v^s \leftarrow \max(\tau_v - C_v, 0) \quad \forall v$ ▷ structural excess
- 2: $\mathcal{D} \leftarrow \{v \mid \bar{W}_v \geq W_{\min} \wedge E_v^s > 0\}$ ▷ sorted by (E_v^s, v) desc.
- 3: $S_v \leftarrow C_v - \tau_v \quad \forall v$ ▷ declared spare capacity
- 4: $\mathcal{R} \leftarrow \{v \notin \mathcal{D} \mid S_v > 0 \wedge \tau_v > 0\}$
- 5: **if** $\mathcal{D} = \emptyset \vee \mathcal{R} = \emptyset$ **then return** $\emptyset$
- 6: $\mathcal{M} \leftarrow \emptyset$
- 7: **for all** $d \in \mathcal{D}$ **do**
- ▷ Compute migration target
- 8: $\hat{E}_d \leftarrow \tau_d \cdot \frac{\bar{W}_d - \bar{W}_d}{\Delta t}$ ▷ inferred excess
- 9: $\gamma_d \leftarrow (\tau_d + \max(\hat{E}_d, 0)) / \tau_d$ ▷ incoming-load scale
- 10: $\sigma_d \leftarrow \min(\max(\hat{E}_d, E_d^s), \alpha \cdot E_d^s)$ ▷ migration target
- 11: **if** $\sigma_d \leq 0$ **then continue**
- ▷ Select donor's own clients to migrate
- 12: $\mathcal{A}_d \leftarrow \{a \mid n_{d,a} > 0\}$ ▷ donor's validated clients
- 13: $\rho_a \leftarrow \tau_d \cdot n_{d,a} / \sum_{a' \in \mathcal{A}_d} n_{d,a'} \quad \forall a \in \mathcal{A}_d$ ▷ committed-rate load
- 14: $\ell_a \leftarrow \gamma_d \cdot \rho_a \quad \forall a \in \mathcal{A}_d$ ▷ incoming-rate estimate
- 15: Sort $\mathcal{A}_d$ by (ρ_a, a) descending
- 16: $\sigma_{\text{done}} \leftarrow 0$
- 17: **for all** $a \in \mathcal{A}_d$ **do**
- 18: **if** $\sigma_{\text{done}} \geq \sigma_d \vee \rho_a \leq 0$ **then break**
- 19: **if** $r - r'_a < 2R \vee \exists (a, \cdot, \cdot) \in \mathcal{M}$ **then continue** ▷ cooldown; one decision per account
- 20: $v^* \leftarrow \arg \min_{v \in \mathcal{R}, S_v \geq \rho_a} (L_{v,a}, -S_v, v)$ ▷ min latency, max spare, id tie-break
- 21: **if** $v^* \neq \perp$ **then**
- 22: $\mathcal{M} \leftarrow \mathcal{M} \cup \{(a, d, v^*, r)\}$
- 23: $\sigma_{\text{done}} += \ell_a; S_{v^*} -= \rho_a$ ▷ reserve at committed rate
- 24: **return** $\mathcal{M}$

as Mysticeti [5], which remove the certificate layer that Flamingo relies on for its load signals and Byzantine-robust aggregation.

Throughout this section, ties in any deterministic ordering or selection are resolved by identifier (validator or account identity).

4.1 Signal Model and Trust Classification

Every client request carries an account identifier a , a nonce q for exactly-once semantics, an application operation, and the identifier of the account’s assigned validator, all bound by the client’s signature. If the assigned validator is unresponsive, the client may resubmit in RELAY mode to a fallback set of $f+1$ validators derived deterministically for that account.

We extend the block header of validator v in round r with three fields, covered by the header signature and hence bound by its PoA certificate $c_{v,r}$: the block proposal timestamp $t_{v,r}$, recorded by the proposing primary after its workers’ batches are quorum-acknowledged; the average batch queue delay $\bar{W}_{v,r}$, which measures how long a sealed batch waits before broadcast; and the per-account transaction count $n_{v,r,a}$, the number of NORMAL-mode requests from account a in v ’s batches of round r . Relay copies and requests whose credential names a validator other than v are excluded from $n_{v,r,a}$, so each request contributes to exactly one account at one validator. Window aggregates discard duplicate (a, q) pairs to prevent replay requests from raising any count. Algorithm 1 derives its inputs from these fields, aggregated over the tracking window. The fields differ in how they can be trusted, and we classify them into three signal classes below.

- **Validated DAG facts:** the per-account transaction count $n_{v,a}$ records how many transactions from account a were included in validator v ’s certified batches. These counts are recomputed from certified batch contents. Every validator holding the batches can verify them, and a validator cannot inflate these counts without actually proposing and disseminating the corresponding transactions.
- **Clock-derived signals:** we assume bounded clock skew, the clocks of honest validators differ by at most a known bound δ . This assumption is needed neither for the safety nor liveness of the protocol; it affects only the magnitude of the estimates below. The throughput and queue-delay estimates are computed over the evaluation interval Δt (Figure 4). The throughput τ_v is the total certified transaction count across v ’s batches in the window, divided by Δt . An evaluation whose Δt is not positive is skipped and recurs at the next anchor.
- **Self-measured signals:** the average batch queue delay $\bar{W}_v$ is the mean of the per-round values $\bar{W}_{v,r}$ over the window, and the baseline $\tilde{W}_v$ is an exponential moving average of these window means across successive evaluation rounds; it smooths transient spikes. Both are self-measured, but each value is bound by its certified block header, so a validator cannot equivocate about its own reports across peers. These values affect only how much load a validator sheds, never the safety of assignments. Two further inputs are fixed in the committed log: the capacity C_v , which declares v ’s safe sustained throughput and is committed once per epoch (we assume an honest validator sets C_v below its saturated throughput, so genuine overload manifests as $\tau_v > C_v$), and the account-validator latency $L_{v,a}$, measured by the client owning a and published as a committed record. Evaluations use the latest $L_{v,a}$ committed before the window opens.

4.2 Load Balancing Algorithm

Algorithm 1 takes as input the signals defined in §4.1 and outputs a set of migration decisions $\mathcal{M} = \{(a, v_{\text{old}}, v_{\text{new}}, r)\}$, each proposing

to move account a from its current validator v_{old} to a new validator v_{new} at anchor round r . The goal is to identify overloaded validators, select accounts to migrate, and assign them to validators with spare capacity. These decisions do not change any assignment directly. A migration takes effect only when $2f+1$ validators produce matching decisions and the receiving validator commits its acceptance.

Donors and receivers (lines 1–5). A validator is a *donor* (overloaded) when two conditions hold: its certified throughput exceeds its declared capacity, and its average queue delay exceeds a threshold $W_{\min}$ that filters transient noise. The first condition grounds overload in validated data; the second prevents jitter from triggering migration. A non-donor with spare capacity and positive throughput is a *receiver*. The positive-throughput rule excludes idle validators whose spare capacity cannot be verified from their DAG participation alone. If either set is empty, no migration occurs.

Migration target (lines 8–11). For each donor d , the algorithm estimates how much load to shed using two signals. The *structural excess* E_d^s captures certified throughput already observed above capacity. The *inferred excess* $\hat{E}_d$ captures arrival pressure not yet visible in throughput: following the standard fluid view of queues [36], sustained queue growth reflects arrivals exceeding service, so the per-window growth rate scaled by throughput estimates the surplus. This is an estimate, since a saturated validator cannot expose its true arrival rate through certified throughput alone. The migration target takes the larger signal and caps the result at α times the structural excess, where $\alpha \geq 1$ controls aggressiveness. The cap ties migration to validated data: even if a donor inflates its self-reported queue delay, it cannot force migration beyond what its certified throughput justifies.

Account selection and receiver assignment (lines 12–23). Candidates are the donor’s accounts in the tracking window, taken heaviest first (lines 12–15). Each account has two load estimates, one for each side of the migration. The *committed rate* ρ_a splits the donor’s certified throughput by the account’s share of validated transactions; since it is fully verifiable, it is what the receiver reserves. The *incoming-rate estimate* $\ell_a = \gamma_d \cdot \rho_a$ scales ρ_a up because a congested donor’s committed rate understates its arrivals; it is what the donor credits toward its target σ_d (line 23). Since $\gamma_d \geq 1$, each account contributes at least ρ_a toward σ_d , favoring conservative shedding; any residual overload is reconsidered at the next anchor. An account is skipped if its assignment changed within the last $2R$ rounds, which prevents rapid back-and-forth migration, or if it already has a decision in this invocation (line 19). Otherwise, it goes to the receiver with the lowest latency $L_{v,a}$ among those with at least ρ_a spare capacity, and is skipped if none qualifies.

The donor classifier uses queue delay rather than raw batch latency because queue delay is the component that grows under sustained overload, regardless of the root cause (e.g., excess client submissions, outbound broadcast contention, or slow peer acknowledgments). The correct action is the same: reduce the donor’s load by migrating accounts away.

Stratus [25] uses an analogous end-to-end *stable time* signal for a related purpose; Flamingo instead decomposes batch confirmation time into queue delay (the overload-sensitive component) and quorum latency (which varies substantially under network imbalance and bandwidth heterogeneity, regardless of overload). Splitting the

Algorithm 2 Migration lifecycle at an honest validator u

▷ At evaluation anchor $r = kR$

- 1: **for all** decisions $(a, v_{\text{old}}, v_{\text{new}}, r)$ output by Alg. 1 **do**
- 2: send $m_u = \langle \text{MIGRATE}, a, v_{\text{old}}, v_{\text{new}}, r \rangle_{\sigma_u}$ ▷ at most one per (a, r)
- ▷ Evidence and activation (Π^+)
- 3: **upon** collecting $2f+1$ matching decisions Π : submit as EVIDENCE record
- 4: **upon** first commit of Π at round r_c : mark Π *live* until activated or until committed round $r_c + T_\Pi$
 Π is *eligible* while $v_{\text{old}} = v_a \wedge r > r'_a$
- 5: **if** Π is live and eligible **and** $v_{\text{new}} = u$ **then**
- 6: submit $\text{acc} = \langle \text{ACCEPT}, a, v_{\text{old}}, v_{\text{new}}, r, D(\Pi) \rangle_{\sigma_{v_{\text{new}}}}$
- 7: let $\Pi^+ = (\Pi, \text{acc})$ ▷ evidence + receiver acceptance
- 8: **upon** commit of acc while Π live and eligible: $A(a) \leftarrow (v_{\text{new}}, r)$
- 9: **upon** expiry of Π unactivated: void Π and exclude v_{new} from receiver selection for a for the rest of the epoch
- ▷ Relay fallback
- 10: **upon** relay copy (a, q) : admit iff
 - (1) $u \in F(a, q)$,
 - (2) client signature and credential valid,
 - (3) credential matches $A(a)$,
 - (4) q is the next uncommitted nonce,
 - (5) no pending fallback copy for a at u ,
 - (6) no effective fallback request from a in the preceding R_f committed rounds
- 11: **upon** admission: reserve a locally until nonce q commits ▷ committed copy leaves $A(a)$ unchanged, excluded from load counters

signals lets us isolate the overload component cleanly and avoid misclassifying network artifacts as overload.

Robustness to misreporting. A Byzantine validator v can misreport its queue delay $\tilde{W}_v$ or capacity C_v . The algorithm limits the damage in each case. If v inflates C_v to appear as a receiver and later fails to accept, the migration protocol (§4.3) protects the affected accounts: an assignment changes only after the receiver's own acceptance is committed, and if the receiver never accepts, the evidence expires and v is excluded from receiver selection for that account for the rest of the epoch. If v deflates C_v or inflates $\tilde{W}_v$ to appear as a donor, the migration target is capped at $\alpha \cdot E_v^s$, which depends on validated throughput and the epoch-committed capacity. Even with $C_v = 0$, the migratable load is bounded by τ_v , the traffic v actually disseminated. The counting rule also limits candidates to v 's own accounts, so a faulty donor cannot redirect another validator's clients. An inflated queue delay raises both $\hat{E}_d$ and γ_d . The first is capped at $\alpha \cdot E_d^s$. The second makes each migrated account count more toward the donor's own target, so the donor reaches its target after shedding fewer accounts. In both cases, the load placed on receivers stays within what they declared.

A genuinely slow validator may produce few certificates, making its computed spare capacity misleadingly large. An idle honest validator has $\tau_v = 0$ and is excluded by the receiver rule even though it could be a good receiver. The receiver test cannot distinguish these two cases. Flamingo bounds the damage through the acceptance requirement and relay fallback. A preventive receiver-health test, such as requiring a minimum certificate count in the window, is compatible with our design; we leave it to future work.

4.3 Migration Protocol

Algorithm 2 summarizes the migration lifecycle: (1) decision signing, (2) evidence formation, (3) migration activation, and (4) relay fallback. We discuss each step and state the correctness properties.

We write $A(a) = (v_a, r'_a)$ for account a 's current assignment and the anchor round of its latest effective migration, initially $(v_0, 0)$,

where v_0 is the validator derived deterministically from the account identifier during Sybil-resistant admission. A committed object is *effective* if it changes protocol state at its position in the committed log: an effective migration updates $A(a)$, and an effective fallback request is the first valid committed copy of a request, the one that exactly-once semantics executes. Any other committed copy is a *no-op* that changes no state.

(1) Decision signing (lines 1–2). At each anchor round $r = kR$, a validator running Algorithm 1 may sign a migration decision $m_i = \langle \text{MIGRATE}, a, v_{\text{old}}, v_{\text{new}}, r \rangle_{\sigma_i}$. An honest validator signs at most one decision per (a, r) . Since R , w , and τ are fixed within the epoch, the anchor r uniquely identifies the tracking window $\mathcal{W}_r = \{r - \tau - w + 1, \dots, r - \tau\}$.

(2) Evidence formation (lines 3–4). A set Π of $2f+1$ matching decisions from distinct validators forms *migration evidence*, submitted to the committed log. Once committed at round r_c , the evidence is *live* until activated or until T_Π committed rounds pass, and *eligible* only while the account is still assigned to v_{old} . The expiry bound T_Π prevents stale evidence from lingering indefinitely.

(3) Migration activation (lines 5–9). An assignment changes only with the receiver's participation. When an honest validator observes committed, live, eligible evidence Π naming it as v_{new} , it submits an acceptance record acc (line 6); its digest $D(\Pi)$ binds acc to this specific evidence and prevents replays. Once acc commits while Π is still live and eligible, every honest validator applies $A(a) \leftarrow (v_{\text{new}}, r)$, and $\Pi^+ = (\Pi, \text{acc})$ becomes the account's new assignment credential. No client signature is required, so a client cannot veto a migration. Until activation, the account stays with v_{old} ; a request that still reaches v_{old} afterwards is a no-op and is resubmitted under Π^+ . If the receiver stays silent, Π expires after T_Π committed rounds, and every honest validator excludes v_{new} from receiver selection for a for the rest of the epoch.

Two safeguards bound the client's exposure when its assignment changes without its participation: the client's committed latency records constrain where it can be placed, and the installed receiver was vouched for by at least $f+1$ honest validators who evaluated validated counts and committed capacities.

(4) Relay fallback (lines 10–11). The assigned validator is the normal route, not the only admission point. If the client receives no reply before its timer expires, it resubmits the same request in RELAY mode to its fallback set $F(a, q)$ of $f+1$ validators, derived deterministically for the account (§4.1) so that at least one member is honest. An honest member admits the copy only under conditions (1) through (6) of Algorithm 2 at line 10. Two of them bound the fallback traffic: an honest relay holds at most one pending copy per account (5), and an account has at most one effective fallback request every R_f committed rounds (6). The committed copy is ordered and executed like any other request but does not change the assignment and is excluded from load counters. Several members of $F(a, q)$ may commit copies of the same request; exactly-once semantics on the nonce makes only the first valid committed copy effective, and every later copy is a no-op.

A Byzantine client may invoke the fallback without contacting its assignee. Honest validators reject copies from outside $F(a, q)$, and after commitment, exactly-once semantics and the R_f gate reject further copies. Condition (6) counts only effective requests: committed duplicates that are no-ops neither reset the gate nor

block future admission, so replaying past requests cannot censor a client’s fallback path.

Correctness. We briefly state the correctness of our migration protocol using three properties.

P1: Assignment agreement. For any committed-log prefix, honest validators derive the same effective assignment for every account.

The committed admission record fixes the same initial assignment for each account. Every subsequent transition is triggered by a committed object: an EVIDENCE record, an acceptance record, or an expiry deadline measured in committed rounds. Two honest validators that have processed the same committed prefix observe the same objects at the same positions and evaluate the same deterministic conditions, so they apply the same transition, or both treat the object as a no-op. Induction over the log gives agreement. P2: Liveness. If a correct client submits a valid request with its next nonce and a current assignment credential, the request eventually takes effect.

The underlying consensus protocol provides eventual reliable communication between a correct client and honest validators, and ensures that a valid request submitted by an honest validator is eventually committed. Flamingo changes only how requests are admitted: a normal request enters through the assigned validator, and a relay copy enters through $F(a, q)$ under conditions (1) through (6). It therefore suffices to show that an honest validator eventually admits the request.

If the assignee is honest, it admits and submits the request, and base-protocol liveness commits it. Otherwise, $F(a, q)$ contains an honest validator, and eventual reliability delivers the relay copy to it. If that validator already holds a pending fallback copy for the account, that copy is eventually committed by base-protocol liveness. Otherwise, once the previous effective fallback request leaves the R_f window, conditions (1) through (6) are satisfied and the copy is admitted; the gate may delay fallback service by up to R_f committed rounds per request but does not permanently block it. Since only effective fallback requests occupy the gate, replayed no-ops cannot postpone reopening indefinitely. The base protocol eventually commits at least one admitted copy; the first valid committed copy takes effect and later duplicates are no-ops. P3: Bounded fallback admission. The fallback path cannot inject unbounded extra traffic: each request yields at most $f+1$ valid copies, at most one committed copy is effective, and an account’s effective fallback requests are at least R_f committed rounds apart.

Only the $f+1$ members of $F(a, q)$ can submit valid fallback copies of a request, and exactly-once semantics makes at most one committed copy effective. Each honest relay admits at most one pending copy per account, and the R_f gate permits at most one effective fallback request per account per interval. Under the admission bound B_{adv} , honest relay validators admit at most $(f+1)B_{adv}$ fallback copies from Byzantine-client accounts per R_f -round interval. Fallback traffic from correct clients with a faulty assignee is their ordinary workload: each request produces at most $f+1$ relay copies, rate-limited by the same gate. Byzantine validators may inject additional copies into their own batches; such copies lie outside the honest-admission bound.

Censorship by the assignee. A Byzantine assignee may silently throttle its clients. Affected clients can still submit requests through

the relay fallback, but only at the rate the R_f gate allows, and their zero validated counts exclude them from every candidate set in Algorithm 1. However, each effective relay request is a committed fact visible to all honest validators. If an account accumulates enough consecutive effective relay requests in the committed log, this pattern signals that its assignee is not serving it. A deterministic threshold on this count, evaluated identically by all honest validators, could trigger a forced migration through the same evidence-and-acceptance lifecycle, reassigning the account without relying on Algorithm 1’s candidate selection. The client can also publish a high latency record $L_{v,a}$ for the censoring validator, steering future placement away from it at epoch reconfiguration. We leave both extensions to future work.

5 EXECUTOR LEVEL LOAD BALANCING

The previous section focused on load balancing at the validator level. This section addresses load balancing at the executor level, where a committed batch from a validator must be distributed among m executor workers, each managing a distinct shard of the state. While scaling the execution layer by distributing execution responsibilities across workers can enhance performance, it presents two main challenges. First, if the transaction access pattern is skewed, workers may end up processing unequal numbers of transactions, negatively affecting the overall performance. Second, processing distributed transactions that access records across multiple workers can be challenging. Moreover, adding more workers tends to increase the proportion of distributed transactions due to the finer data sharding.

Flamingo adopts a *data fusion* model [41]: each transaction is routed to a designated master executor worker that pulls any required remote inputs, executes locally, and installs all writes in its own store without writing state back to the originating executor workers. This reduces the cost of a distributed transaction from a round trip to a single one-way transfer, and subsequent transactions assigned to the same master that access recently fused state incur no transfer at all. We assume read/write sets are statically extractable prior to execution, which holds for Move-based transaction formats. A replicated BFT executor must execute transactions in the consensus-determined order, and nodes may operate different numbers of executor workers, so the scheduler cannot reorder transactions within a batch to improve locality.

A centralized scheduler would become a bottleneck and face the difficulty of synchronizing partition state across workers. Instead, when a batch is committed, each validator worker broadcasts the *full batch* to all m executor workers, and each executor worker independently runs Algorithm 3 on the full batch, then executes its assigned transactions in the consensus-determined order.

This design requires each executor worker to ingest the entire committed stream, covering the records owned by all executor workers, rather than a b/m sub-batch; relative to a sub-batch design that delivers $O(b/m)$ data per executor worker, per-worker LAN ingress grows by a factor of m . This in-cast is affordable because the validator workers, primary, and executor workers are mutually trusted components within a single node, typically co-located within the same physical node or the same datacenter with high-bandwidth LAN interconnects (e.g., 10 Gbps), whereas a validator worker’s throughput is already capped by the lower WAN egress

bandwidth used for inter-validator dissemination. The full-batch broadcast is therefore absorbed whenever the per-executor LAN ingress bandwidth exceeds the worker WAN egress bandwidth by at least a factor of m . Since intra-datacenter LAN bandwidth is typically at least an order of magnitude higher than WAN bandwidth, this condition holds for the executor scales we target ($m \leq 10$).

Algorithm 3 presents the execution load balancing in Flamingo. The algorithm processes transactions sequentially in the consensus-determined order. For each transaction, it tallies votes from state ownership (lines 4–6): each key in the read/write set votes for the executor worker that will own it after previous transactions are executed. The executor worker is then selected in one of the following ways.

- (1) **Fast path** (lines 8–9): If the entire read/write set maps to one executor worker with available capacity, assign the transaction to that executor worker (zero remote reads).
- (2) **Affinity path** (lines 10–11): Otherwise, choose the highest-vote executor worker with sufficient capacity, breaking ties by load, then by ID for determinism.
- (3) **Fallback** (lines 12–13): If all candidates are full, assign to the least-loaded executor worker. The per-executor capacity (line 1) is bounded by $\lceil \frac{|b|}{m} (1 + \epsilon) \rceil$ where ϵ is a configurable load imbalance tolerance parameter.

After each assignment, the algorithm updates load counters and rewrites the keys in a batch-local overlay (lines 14–16). Subsequent transactions compute votes against this overlay, ensuring that scheduling decisions are consistent with the evolving execution plan. This prefix-consistent view is also required by executor workers to determine whether to serve or fetch the state.

The overlay can be viewed as a sequence of delta updates over the base partition map, which provides a compact representation of the evolving ownership state. This enables re-execution by reconstructing the intermediate ownership state from the base map and the sequence of assignments, without materializing a full updated partition. Between batches, the final overlay U captures the up-to-date data ownership: it serves as the base partition map for scheduling the next batch, and executor workers use it to verify the data ownership of each subsequent transaction’s read/write set.

The algorithm is order-preserving and deterministic: it processes transactions in consensus-determined order, uses deterministic tie-breaking, and derives all state from the batch and initial partition map. Executor workers with the same inputs therefore produce identical assignments without a central dispatcher, and nodes agree on execution regardless of their worker count.

6 EVALUATION

The goal of our evaluation is to quantify how effectively Flamingo mitigates the performance degradation caused by load imbalance in DAG-based BFT protocols. Flamingo performs transaction submission routing and execution scheduling at the ordering and execution layer respectively. We evaluate each layer independently because the two are subject to different bottlenecks and thus require different baselines. At the ordering layer, we compare Flamingo against vanilla Narwhal [20] to quantify both the performance gain under skewed workloads and the overhead under balanced

Algorithm 3 Load-Aware Transaction Scheduling in Flamingo

Input: batch b , executors m , partition map P , imbalance tolerance ϵ

Output: assignment list $\mathcal{A}$ of (tx, e) pairs

```

1:  $cap \leftarrow \lceil \frac{|b|}{m} \cdot (1 + \epsilon) \rceil$  ▷ per-executor capacity
2:  $U \leftarrow P$ ;  $load[1..m] \leftarrow 0$ ;  $\mathcal{A} \leftarrow []$  ▷  $U$ : mutable overlay on  $P$ 
3: for all  $tx \in b$  do ▷ in consensus order
   ▷ Tally ownership votes over the overlay
4:    $votes \leftarrow \emptyset$ 
5:   for all  $k \in RW(tx)$  do
6:      $votes[U[k]] += 1$ 
   ▷ Select an executor worker
7:    $C \leftarrow \{e : votes[e] > 0 \wedge load[e] < cap\}$ 
8:   if  $|votes| = 1 \wedge C \neq \emptyset$  then
9:      $e^* \leftarrow \text{the element of } C$  ▷ fast path: zero remote reads
10:  else if  $C \neq \emptyset$  then
11:     $e^* \leftarrow \arg \max_{e \in C} (votes[e], -load[e])$  ▷ affinity path
12:  else
13:     $e^* \leftarrow \arg \min_e load[e]$  ▷ fallback: all candidates at capacity
   ▷ Record the assignment and update the overlay
14:    $\text{append}(tx, e^*) \text{ to } \mathcal{A}$ ;  $load[e^*] += 1$ 
15:   for all  $k \in RW(tx)$  do
16:      $U[k] \leftarrow e^*$ 
17: return  $\mathcal{A}$ 

```

workloads. At the execution layer, the closest scale-out baseline is PilotFish [37], which fixes object-to-worker placement at dispatch time and thus offers no load-aware scheduling. We separately measure (1) throughput of Flamingo’s executor with load-aware scheduling disabled, evaluated by replaying committed batches, and (2) the isolated scheduling cost versus remote-read trade-off against Hermes-style prescient routing.

At the ordering layer, Flamingo augments Narwhal block headers, and hence the certificates over them, with additional metadata: per-account validated transaction counts, the block proposal timestamp, and the average batch queue delay. The load signals therefore piggyback on the DAG, so a validator computes and verifies its decisions from certificates with no dedicated metric-exchange round. The evaluated prototype implements this signal-and-decision path, which §6.2.6 additionally exercises under validators that misreport their self-declared signal; the migration-commit lifecycle remains a protocol-level Byzantine-safety mechanism whose guarantees are established analytically in §4.3. At the execution layer, Flamingo runs the full-batch decentralized scheduling design of §5.

6.1 Experimental Setup

Experiments were conducted on CloudLab [22] using x1170 nodes, each equipped with an Intel Xeon E5-2640 v4 (10 physical cores, 20 hardware threads), 64 GB DDR4 ECC memory, and a 480 GB SSD, connected via up to 25 Gbps links. WAN conditions (bandwidth and 100 ms RTT) are shaped by the CloudLab profile. Within each validator, we cap bandwidth at 300 Mbps for the primary and 600 Mbps for the worker to mirror a scaled-out DAG-based deployment, where operators prioritize bandwidth for consensus traffic. System components are pinned to disjoint CPU sets at the hardware-thread level: 10 logical CPUs for workers, 6 for the primary, and 2 for clients, with the remainder reserved for the OS. Worker batch storage uses an in-memory HashMap-backed store to isolate network and consensus bottlenecks from disk I/O, which is orthogonal to our load-balancing contribution; the primary’s storage remains RocksDB-backed.

We use the SmallBank benchmark with 1 M users. Each validator is paired with one benchmark client that simulates a geo-distributed region. Account identifiers are partitioned into equal contiguous ranges, and the identifier-derived initial assignment (§4.3) maps each range to the region’s co-located validator; each client submits transactions on behalf of its region’s accounts at a fixed rate. The user for each transaction is selected according to the configured skewness, and transactions are padded with zeros up to the configured size. To prevent TCP backpressure from both throttling the client sending rate and inflating tail latency, the client buffers requests in an unbounded channel.

The baseline validator configuration uses 4 validators each running a single worker, a 500 KB batch size, a 1 KB block size, 200 ms maximum batch and header delay, and a transaction size of 512 B. The load balancing algorithm (Algorithm 1) is invoked every $R=60$ rounds over a $w=30$ -round tracking window. We set the migration-amount cap $\alpha = 4$; larger values shed more load per invocation but risk ping-pong oscillation between donor and receiver. We set the donor queue-delay threshold $W_{\min} = 30$ ms to filter out network jitter; $W_{\min}$ is the minimum queue-delay threshold a validator must exceed to be classified as a donor. Each validator computes its capacity C_v (§4) from its egress bandwidth, since broadcast fan-out saturates the network before CPU or disk in a scaled-out DAG-based BFT deployment. Concretely, C_v is the sustainable egress throughput of the validator’s broadcast:

$$C_v = \left\lfloor \frac{BW}{(s + 44)(n - 1)} \cdot 0.8 \right\rfloor, \quad (1)$$

where BW is the validator’s egress bandwidth, s is the transaction payload size in bytes, and the constant 44 bytes accounts for per-transaction wire overhead (40 B TCP/IP header plus 4 B length-prefix codec overhead). The $(n-1)$ factor reflects that each worker broadcasts every transaction to all other $n-1$ validators, and the 0.8 multiplier leaves headroom for control traffic and short bursts. Other components (e.g., primary CPU or storage) may become limiting in setups with narrower WAN links, and C_v can be adjusted accordingly.

The per-account-per-validator RTT $L_{v,a}$, used by the migration policy to select a receiving validator for a shed account, is 100 ms for remote-region submissions and 0 ms locally. We use a garbage collection (GC) depth of 600 rounds to prevent blocks and certificates needed by slow validators from being garbage collected before they catch up. Each run lasts 15 minutes unless stated otherwise. Clients first submit a balanced workload for 4 minutes (the *balanced phase*) and then switch to the configured skewness for the remainder (the *imbalanced phase*). Committed throughput over time is computed as the total number of committed transactions across validators binned at 10 seconds granularity, smoothed with a 3-bin moving average. Latency is measured as the interval between transaction submission and the time at which the transaction is committed by the consensus protocol, tracked via sample transactions under high load.

6.2 Validator Level Load Balancing

We evaluate validator level load balancing along six axes: submission rate (§6.2.1), workload skewness (§6.2.2), per-validator bandwidth heterogeneity (§6.2.3), number of validators (§6.2.4), adaptivity to shifting hotspots (§6.2.5), and robustness to misreporting validators (§6.2.6). Our goal is to demonstrate that Flamingo (1) rebalances skewed workloads to recover throughput and latency, (2) distributes the load efficiently across heterogeneous validators, (3) introduces negligible overhead when no rebalancing is required, (4) scales with the number of validators, and (5) remains effective when validators misreport their self-declared load signal.

We define workload imbalance as 60% or 90% of the total load being directed at 25% of the validators, and validator heterogeneity as up to f or $f+1$ validators being bandwidth-capped at 200 Mbps. The highest submission rate we evaluated, 110 ktps, is roughly 76% of the aggregate service throughput, which allows us to observe whether Flamingo can dissipate the backlog built up during the imbalanced phase in a reasonable time.

6.2.1 Impact of Submission Rate. We evaluate Flamingo under four submission rates: 20, 50, 80 and 110 ktps, with the imbalanced phase skewness fixed at 90%. Figure 5a shows throughput (top), mean latency (middle), and cumulative account migrations (bottom). During the balanced phase, Flamingo matches the baseline on both throughput and latency, confirming that the load-balancing algorithm imposes negligible overhead when no migration is required. The brief dip and rebound in the 110 ktps run before the phase change is a consensus-ordering artifact in which one validator’s certified batches are temporarily not referenced by leader blocks and are then ordered together once referenced; certification throughput remains flat over this window and no migrations are triggered.

Under a light workload of 20 ktps, even routing 90% of the load to a single validator remains well below its processing capacity, and Flamingo correctly migrates no accounts. At higher rates, the onset of the imbalanced phase reduces performance for both Flamingo and the baseline (which does not employ load balancing techniques). Flamingo, however, dissipates the backlog and recovers throughput and latency, whereas the baseline does not. At 50 ktps, Flamingo operates near the saturation knee of the first validator: an initial burst of migrations rebalances the load, after which no further rebalancing is induced. During the initial phase, the queue-delay differential ($\bar{W}_v - \bar{W}_d$) is positive, causing Algorithm 1 to classify v_0 as a donor; once the system stabilizes, the delay difference becomes small or negative. At 80 and 110 ktps, both E_d^s and $\bar{E}_d$ are larger, driving more migrations and a longer backlog drain. Convergence time grows monotonically with submission rate, as the cumulative migration curves make explicit.

6.2.2 Impact of Workload Skewness. Figure 5b fixes the submission rate at 110 ktps and varies skewness from 60% to 90% to 99%. Higher skewness concentrates more load on fewer validators, raising the overload on the donor and thus the migration target. As a result, Flamingo migrates more accounts and takes longer to drain the backlog: the 99%-skew run converges roughly two minutes later than the 60%-skew run, with the 90%-skew case in between.

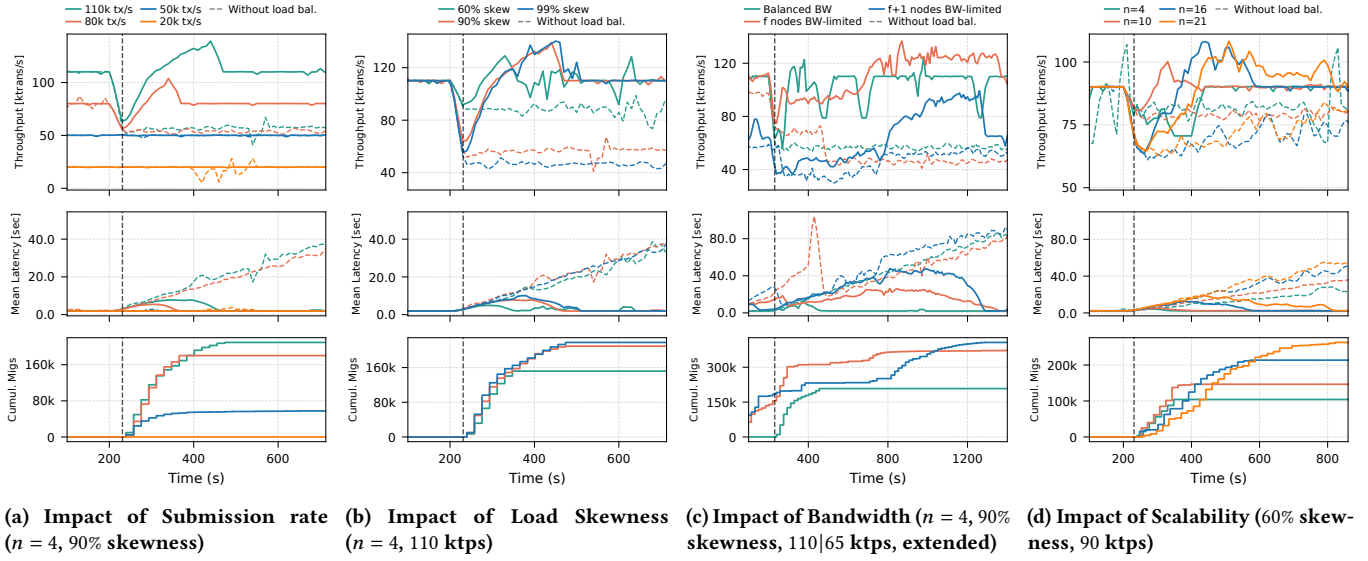


Figure 5: Validator level load balancing evaluation

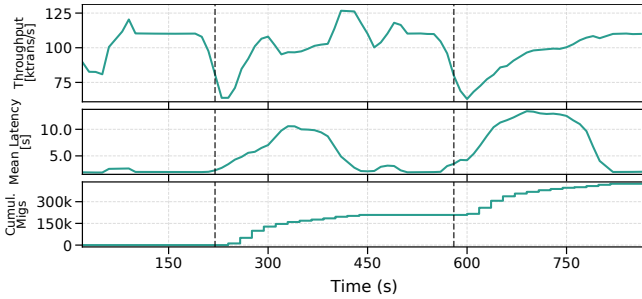


Figure 6: Adaptivity of Flamingo under a shifting hotspot. Balanced phase (0–4 min); first imbalanced phase with hotspot on v_0 (4–10 min); second imbalanced phase with hotspot shifted to v_1 (10–15 min). Flamingo issues a second migration burst at the 10-minute shift and recovers throughput in both phases.

The cumulative migration curve scales accordingly, and the recovered throughput under all three skewness settings approaches the balanced-phase level, whereas the baseline remains degraded throughout the imbalanced phase.

6.2.3 Impact of Bandwidth Heterogeneity. We next evaluate Flamingo under heterogeneous validator bandwidth. A *slow validator* is one whose per-worker egress bandwidth is capped at 200 Mbps (instead of the 600 Mbps default); its primary-side bandwidth is unchanged. This models realistic deployments where a subset of operators runs on degraded WAN links. We evaluate two configurations: up to f slow validators (which the quorum threshold $2f+1$ can mask) and $f+1$ slow validators (which it cannot). In the first configuration, the submission rate is 110 ktps, while in the second configuration we limit the submission rate to 65 ktps.

Convergence here is paced by the donor condition of Algorithm 1. Queue delay alone never qualifies a validator as a donor: donor qualification and the per-anchor target $\alpha \cdot E_d^s$ are anchored in validated throughput above committed capacity, and a saturated validator’s certified throughput understates its arrival rate (§4), so its structural excess stays small and load is shed through a sequence of bounded migration waves, visible as the staircase in the cumulative-migration curves, with residual overload recurring at later anchors by design. We therefore extend these runs to 25 minutes, keeping the 4-minute balanced phase, so the full multi-wave convergence and the post-recovery steady state fall within the measurement window. The results are presented in Figure 5c.

With f bandwidth-limited nodes, Flamingo begins migrating accounts away from the slow validators immediately, improving throughput during the balanced phase. When the imbalanced phase starts, both Flamingo and the baseline degrade; a second migration wave from the overloaded v_0 restores throughput to near the offered rate, and the backlog then drains gradually through small jitter-triggered residual waves, with mean latency returning to the balanced-phase level in the later part of the run.

With $f+1$ bandwidth-limited nodes, forming a quorum requires $2f+1$ validators and the $(2f+1)$ -th fastest validator is itself slow, so no migration can restore the balanced-bandwidth commit rate. The slow validators still satisfy the donor condition of Algorithm 1, so Flamingo sheds accounts toward the fast validators from the start of the run; a second migration wave mid-run sheds further load, after which committed throughput temporarily exceeds the offered 65 ktps while the backlog drains and mean latency returns near the balanced-phase level by the end, whereas the baseline’s latency grows for the remainder of the run.

6.2.4 Scalability with Number of Validators. Figure 5d evaluates Flamingo across $n \in \{4, 10, 16, 21\}$ validators at a fixed offered rate of 90 ktps. Unlike the preceding experiments, increasing n reduces

each validator’s modeled capacity because every worker broadcasts each batch to $n-1$ peers (Eq. 1). Consequently, 110 ktps corresponds to approximately 76% of the modeled aggregate capacity at $n=4$ but approximately 97% at $n=21$, leaving almost no spare capacity for absorbing migrated load and draining the backlog. We therefore fix the rate at 90 ktps, approximately 80% of the minimum aggregate capacity across the evaluated committee sizes, so that every configuration retains comparable headroom and the observed differences are attributable to committee size rather than saturation.

During the balanced phase, when no migrations are required, Flamingo’s overhead is negligible and its performance matches the baseline across all network sizes. When the imbalanced phase begins, 60% of the load is concentrated on 25% of the validators (i.e., {1, 3, 4, 6} donors respectively), and all configurations degrade. Flamingo recovers in every configuration: throughput briefly exceeds the offered rate while the backlog drains and then settles back at 90 ktps, and mean latency returns to the balanced-phase level. Recovery takes longer, and cumulative migrations grow, as the network size increases.

As n grows, the per-donor skewed load τ_d shrinks (the total rate is split across more donors, roughly $0.25n$ of them), and the per-donor capacity C_d also shrinks because the broadcast fan-out factor $(n-1)$ in Eq. 1 reduces each worker’s sustainable throughput. Both terms are $O(1/n)$, so in the overload regime ($\tau_d > C_d$) their difference $E_d^s = \tau_d - C_d$ is $O(1/n)$ as well. The migration cap $\alpha \cdot E_d^s$ shrinks accordingly, so Flamingo sheds less load per invocation at larger n , which explains the longer convergence time and also suppresses ping-pong.

6.2.5 Adaptivity to Shifting Hotspots. The preceding experiments show that Flamingo converges once after a single static workload shift. Hotspots can also move over time, and the load balancer must keep up with them rather than lock in the first assignment. To evaluate this, we run a three-phase workload at 110 ktps and 90% skew on $n=4$ validators: a balanced phase (0–4 min) matching the earlier setup, a first imbalanced phase concentrating load on v_0 (4–10 min), and a second imbalanced phase in which the hotspot shifts to v_1 (10–15 min). Figure 6 shows that Flamingo migrates accounts away from v_0 at the onset of the first shift and recovers throughput, as in §6.2.1. When the hotspot moves to v_1 at 10 minutes, the queue-delay signal on v_1 exceeds $W_{\min}$, triggering a second migration burst that re-routes accounts toward the now-underloaded v_0 and the remaining validators; throughput recovers a second time. The no-load-balancing baseline remains degraded across both imbalanced phases. This confirms that Flamingo’s load balancer responds to ongoing load shifts rather than acting as a one-shot rebalancer.

6.2.6 Robustness to Misreporting Validators. We stress the two self-reported inputs of Algorithm 1 by reusing the 110 ktps run from §6.2.1 and having the last validator falsify one signal per run: the queue delay $\bar{W}_v$ or the capacity C_v , inflated or deflated. Under a balanced workload, no attack degrades performance (Figure 7a), since no validator qualifies as a donor and the misreports have nothing to act on.

Under 90% skew, three of the four attacks still track the honest baseline (Figure 7b). Deflating $\bar{W}_v$ or C_v only removes the misreporting validator from its own donor or receiver role. Inflating $\bar{W}_v$ raises

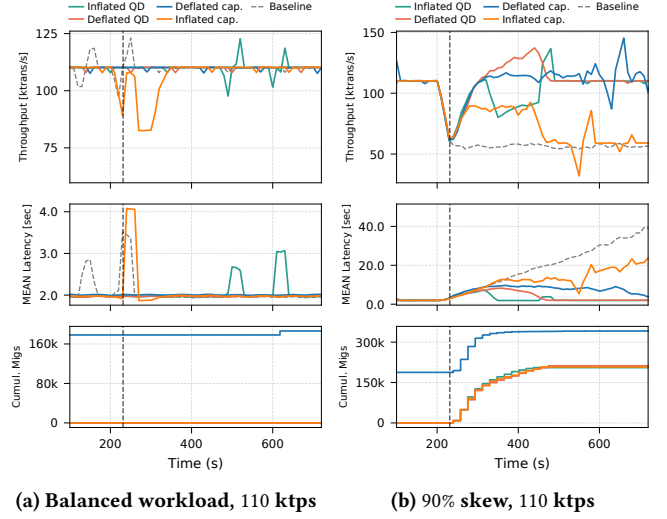


Figure 7: Robustness to misreporting. The last validator falsifies one signal per run. Under a balanced workload (a), no attack degrades performance. Under skew (b), only inflated capacity, the one signal unverifiable at commit time, has an effect: one bounded migration burst and degraded performance for the remainder of the epoch.

the donor’s target, but that target is capped at $\alpha \cdot E_d^s$, computed from validated throughput and cannot be inflated by self-reports.

Inflated capacity behaves differently. A capacity declaration is a commitment about future service, and no certified evidence contradicts it at commit time (§4.2). The misreporting validator triggers a single migration burst, still capped per anchor by each donor’s $\alpha \cdot E_d^s$, after which the migration curve flattens. Throughput and latency stay degraded for the rest of the run: the inflated capacity hides the validator’s structural excess, so it never re-qualifies as a donor within the epoch.

Two safeguards bound the harm. Its magnitude is capped by the donors’ per-anchor targets, and its duration is capped by the epoch boundary, where capacity is re-committed. Meanwhile, affected clients keep making progress through the relay fallback. Both mechanisms are specified in §4.3 and are established analytically.

6.3 Executor Level Load Balancing

We evaluate the executor level along two axes: a scalability experiment that replays committed batches on CloudLab to measure throughput, and a scheduler microbenchmark that isolates scheduling cost vs. remote-read trade-off against Hermes-style routing [42].

6.3.1 Scalability Experiment. Using the same CloudLab nodes as §6.1, we connect each Narwhal worker to its executor workers via 10 Gbps LAN links, which carries the full-batch broadcast required by our decentralized scheduling design. Each validator worker commits roughly 25 ktps. To stress load imbalance, we skew the initial partition so that 50% of transactions are assigned to the first executor worker. We vary the number of executor workers (and, proportionally, validator workers) from 1 to 8 and measure throughput.

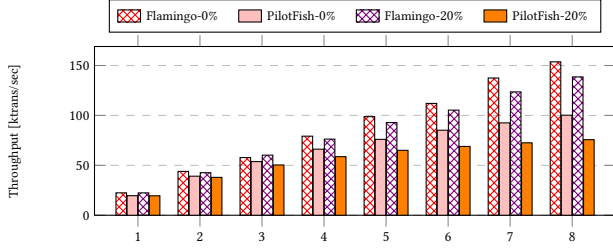


Figure 8: Executor throughput under 50% initial skew. Flamingo vs. PilotFish as executor/validator workers scale from 1 to 8, for single-shard ($\text{dist_tx}=0$) and 20% distributed ($\text{dist_tx}=0.2$) workloads.

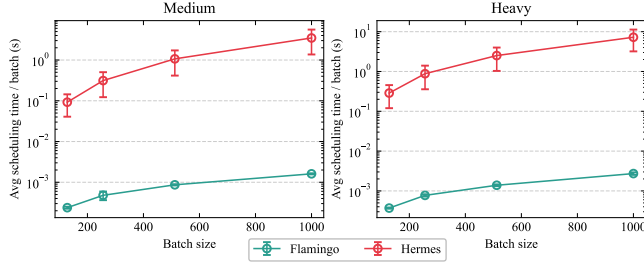


Figure 9: Scheduler runtime per batch as batch size grows (log scale). Flamingo’s single forward pass stays at the millisecond level; Hermes’s iterative rerouting reaches multi-sec range.

The dist_tx parameter controls the fraction of `SendPayment` transactions, which touch two accounts and therefore may become cross-shard; when $\text{dist_tx}=0$ the workload is entirely single-shard. We tune the executor service throughput to be only slightly above the committed rate so that any imbalance directly hurts throughput.

Figure 8 shows the results. Flamingo outperforms PilotFish across all executor worker counts in both workload variants. Both systems improve with scale-out, but Flamingo shows a larger gain because its scheduler redistributes load across executor workers. In the single-shard case ($\text{dist_tx}=0$), throughput scales near-linearly for Flamingo: adding executor workers increases aggregate service capacity with minimal cross-executor coordination, and Flamingo reaches ~ 154 ktps at $m=8$ (about a $6.9\times$ speedup over $m=1$), while PilotFish saturated near 100 ktps. When 20% of transactions are distributed ($\text{dist_tx}=0.2$), scaling becomes sub-linear because each additional executor worker increases the probability that a distributed transaction’s read and write sets span different shards, requiring remote reads and coordination that offset the added parallelism. Nonetheless, Flamingo sustains a consistent advantage, validating the practicality of full-batch decentralized scheduling.

6.3.2 Scheduler Microbenchmark. SmallBank transactions touch at most two accounts, which is too narrow to exercise scheduling quality meaningfully. We therefore use a synthetic key-set workload with controllable transaction size and temporal overlap. Transaction sizes are drawn from three distributions: LIGHT (2 keys with 0.8 probability, 4 with 0.2 probability: 2/0.8, 4/0.2), MEDIUM (2/0.5, 4/0.3, 8/0.2), and HEAVY (4/0.5, 8/0.3, 16/0.2). The *overlap probability* controls how likely a new transaction’s accounts are drawn from a recent history window of previously seen accounts; higher values

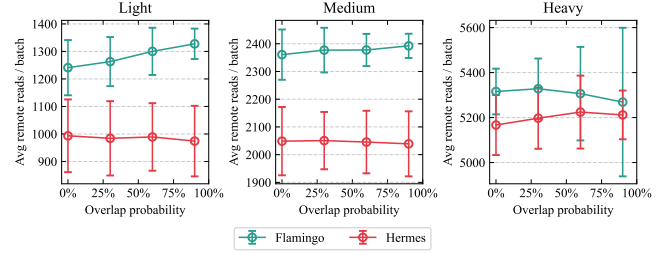


Figure 10: Average remote reads per batch vs. overlap probability across transaction sizes. Error bars span 3 seeds.

create temporal locality where the same accounts recur in bursts. We fix 8 executor workers, 1 M accounts with range partitioning, batch size 1000, and report averages over 5 batches.

We compare against Hermes using a scheduler-level simulator. This choice is deliberate: Hermes’s prescient routing is defined over a reordered batch, and its benefit comes from batch-wide lookahead and rerouting. Reproducing its full-system impact would require fixing to a specific communication model, object size, and migration cost that are orthogonal to the scheduling question. Our simulator fixes the synthetic workload above and measures total remote reads and scheduler runtime. Following Hermes’s simplification, transactions expose explicit read and write sets, and persistent ownership changes are applied through the write set.

Figure 9 shows scheduling time as batch size grows. Flamingo remains at the millisecond level across all batch sizes and transaction types (roughly 2 ms for a batch size of 1000), while Hermes grows into the multi-second range; a gap of approximately three orders of magnitude. This is expected because Hermes optimizes over a permuted batch and performs iterative rerouting to satisfy load constraints, whereas Flamingo makes a single deterministic forward pass that preserves the agreed-upon transaction order.

Figure 10 shows total remote reads as overlap probability varies. Hermes achieves fewer remote reads across all transaction sizes because reordering lets it exploit future locality that Flamingo cannot use. The gap is consistent: roughly 20–30% for light and medium transactions, narrowing for heavy transactions where the larger key sets make most transactions inherently cross-shard regardless of scheduling. Error bars (3 seeds) confirm that the gap is stable, though variance increases with transaction size. Despite this gap, the trade-off favors Flamingo: a moderate increase in remote reads buys a $\sim 1000\times$ reduction in scheduling time.

These results support our design choice: Flamingo recovers much of the locality benefit of prescient routing while remaining cheap enough to run independently on every executor worker.

6.4 End-to-End Evaluation

The preceding sections evaluate the two load-balancing layers in isolation. However, their effects are coupled in a full deployment: validator migration changes where the requests are proposed, and the executor layer must absorb whatever the ordering layer commits. We therefore run the full pipeline, from client submission

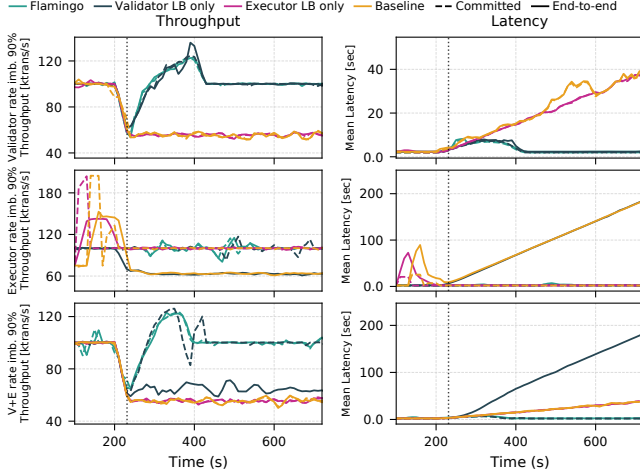


Figure 11: E2E evaluation under validator-level (top), executor-level (middle), and combined (bottom) 90% imbalance. Dashed lines: committed (consensus) performance; solid lines: E2E (committed and executed) performance.

through consensus to execution, and measure both *committed* performance and *end-to-end* (E2E) performance (transactions committed and executed).

We deploy $n=4$ validators, each attached to 3 executor workers, at 110 ktps rate with the two-phase workload from the experimental setup as in §6.2.1. We induce imbalance at 90% in three ways: (i) *validator rate imbalance*, where 90% of submissions target accounts assigned to a single validator; (ii) *executor rate imbalance*, where 90% of transactions touch accounts residing in a single executor worker’s shard; and (iii) both simultaneously. We compare four variants: full Flamingo, Flamingo with validator-level load balancing only, Flamingo with executor-level load balancing only, and the baseline (no load balancing). Figure 11 shows the results.

Under validator rate imbalance (top), the bottleneck lies before commitment, so only the variants with validator-level load balancing recover: full Flamingo and the validator-only variant dissipate the backlog and return committed and E2E throughput to the offered rate, with latency returning to the balanced-phase level. The executor-only variant tracks the baseline: no scheduling decision made after commitment relieves an overloaded ordering path.

Under executor rate imbalance (middle), it inverts. Consensus is unaffected, and all variants sustain the offered committed rate. Without executor-level load balancing, E2E throughput settles well below the committed rate and E2E latency grows as the execution backlog accumulates. Full Flamingo and the executor-only variant keep E2E throughput matched to the committed rate with flat latency, confirming that the executor scheduler resolves an imbalance that is invisible at the consensus layer.

The combined imbalance (bottom) shows the bottleneck shifting from one layer to another. Only full Flamingo restores E2E performance to the offered rate. The validator-only variant resolves the upstream bottleneck but exposes the downstream one: it admits and commits the full rate, but the committed stream lands on the hot executor worker, so a post-commit execution backlog

accumulates and its E2E latency grows even though every transaction commits promptly. The executor-only variant and the baseline remain bottlenecked at the ordering layer throughout. The baseline never reaches the executor bottleneck: its collapsed ordering layer throttles admission to roughly the hot executor’s capacity, so its executors keep up and its latency growth stems from pre-commit queueing at the hot validator instead. The plotted mean latency also understates the baseline’s degradation: latency samples are drawn uniformly across regions while the baseline’s congestion is concentrated in the single hot region, whereas the validator-only variant’s executor backlog delays all regions alike. These results show that the two mechanisms compose without interference and that each layer is necessary for the bottleneck it targets: resolving only one layer moves the bottleneck rather than removing it.

7 RELATED WORK

Flamingo intersects four lines of work: load balancing at the validator (ordering) level, adaptive partitioning and sharding, scheduling of ordered transactions across executors, and the use of time-derived signals under Byzantine faults.

Validator level load balancing. Stratus [25] targets the leader bottleneck of single-leader BFT via a shared mempool and proxy forwarding: overloaded replicas forward excess microblocks to a lightly loaded proxy. Neither mechanism carries over to certified DAG mempools [20]. Every validator is a full-time proposer whose blocks must reference batches processed by its own workers, leaving no idle capacity for proxy duty, and the $2f+1$ availability certificate must guarantee retrievability of batches, which Stratus’s $f+1$ PAB shortcut cannot provide. Routing excess batches through a proxy therefore adds a relay step rather than removing load, and our implementation of this approach did not yield improvements.

Recent DAG-based protocols adapt to underperforming validators at the ordering-role level: Shoal, Shoal++, and HammerHead [3, 53, 59] use leader reputation to exclude slow or crashed leaders, while Sailfish, Mysticeti, Autobahn, and Mahi-Mahi [5, 27, 32, 52] reduce commit latency through multi-leader or uncertified designs. These works reassign consensus roles but leave the client-to-validator assignment, and hence the dissemination load, untouched. Flamingo is complementary: it rebalances client demand over any certified DAG mempool using metadata already in the DAG. Unlike systems that use time-derived signals to determine ordering [34, 35, 63], Flamingo’s header timestamps and queue delays serve only as migration signals and never influence ordering.

Adaptive partitioning and sharding. In trusted deployments, workload-driven partitioners [15, 49, 51, 57] repartition or migrate data in response to workload skew. In untrusted settings, sharded BFT systems such as AHL [21] and SharPer [2] partition data across BFT clusters with a fixed mapping. BrokerChain [31] and Marlin [47] adaptively redistribute load. Marlin balances load across multiple BFT instances by changing data ownership, whereas Flamingo balances load within a single leaderless DAG-BFT instance by migrating client-to-validator assignments and committing migrations through the existing BFT log. Marlin adapts despite Byzantine nodes that may report misleading statistics, using either hypergraph partitioning within a trusted administrative domain or decentralized key-affinity resharding. The distinction is that

Table 1: Distributed transaction execution schemes along four dimensions; b is the batch size per consensus epoch, m the number of executor workers within a node, and a the maximum transaction size (number of states accessed).

System	Bandwidth	Lock time	Sched.	Load Bal.
LEAP [41]	$b, 1 \times$ transfer	Single-trip	per-tx [†]	No
PilotFish [37]	$b/m, 2 \times$ transfer	Round-trip	$O(ba/m)$ [‡]	No
Hermes [42]	$b, 1 \times$ transfer	Single-trip	$O(b^2 a^2 m)$	Yes
Flamingo	$b, 1 \times$ transfer	Single-trip	$O(ba)$	Yes

[†]LEAP decides placement per transaction at submission time rather than scheduling a batch. [‡]Per worker sub-batch; placement is fixed by the object-to-worker partitioning.

Flamingo derives signals from the certified DAG rather than exchanged reports, and requires no separate atomic-commitment protocol for migrations.

Executor level load balancing. Table 1 compares representative distributed execution mechanisms along bandwidth, lock-holding window, scheduling cost, and load balancing. Of these, only Hermes [42] and Flamingo perform load-aware scheduling; Flamingo differs in preserving the committed order and supporting validators with different numbers of executor workers, motivating its full-batch decentralized scheduler with $O(ba)$ per-worker cost (§5). Deterministic databases fix execution order before locking: Calvin [58] pioneered this, Bohm and early write visibility [23, 24] exploit the known order for multiversioning and pipelining, and Aria [43] executes batches deterministically without prior read/write-set knowledge. Their performance still depends on partition quality, and none balances load across a BFT validator’s executor workers. LEAP [41] pioneered what Hermes [42] later termed the look-present approach, migrating each distributed transaction’s state to the executing node but deciding placement per transaction without any view of load. Hermes lifts it to the batch level with load-aware prescient routing, whose reordering of conflicting transactions a replicated BFT executor cannot adopt. PilotFish [37], the closest to our setting as a scale-out engine over an already ordered stream, fixes object-to-worker placement at dispatch time and leaves no handle for load-aware scheduling.

Time-derived signals under Byzantine faults. Pompe [63] assigns each command the median of $2f+1$ signed local timestamps, which directly determines its position in the finalized order. Aequitas and Themis [34, 35] instead avoid timestamp aggregation and derive fairness constraints from replicas’ relative receive orders, using Byzantine-robust agreement and threshold rules to constrain the admissible ordering. In all three systems, these validator observations enter the ordering semantics themselves. Flamingo draws the opposite boundary: header timestamps and queue delays never influence ordering. They serve only as performance signals for migration, filtered by medians and capped by certified throughput, so misreporting can degrade migration quality (§4) but can never affect ordering safety.

8 CONCLUSION

This paper addresses load imbalance across the ordering and execution layers of certified DAG-based BFT protocols. At the ordering layer, Flamingo dynamically rebalances client-validator assignments using load signals derived from the certified DAG, while a committed migration protocol preserves consistent assignments and serviceability under Byzantine faults. At the execution layer, Flamingo uses a deterministic, order-preserving scheduler to balance work across executor workers while minimizing data movement. Our evaluation on Narwhal and Tusk shows that Flamingo recovers performance under workload skew, validator heterogeneity, and shifting hotspots, scales across validators and executor workers, and incurs negligible overhead when load is balanced. End-to-end results further show that the two mechanisms compose effectively and that both layers must be addressed to remove their bottlenecks.

REFERENCES

- [1] Amitanand S Aiyer, Lorenzo Alvisi, Allen Clement, Mike Dahlin, Jean-Philippe Martin, and Carl Porth. 2005. BAR fault tolerance for cooperative services. In *Symposium on Operating Systems Principles (SOSP)*. ACM, 45–58.
- [2] Mohammad Javad Amiri, Divyakant Agrawal, and Amr El Abbadi. 2021. SharPer: Sharding Permissioned Blockchains Over Network Clusters. In *SIGMOD Int. Conf. on Management of Data*. ACM, 76–88.
- [3] Balaji Arun, Zekun Li, Florian Suri-Payer, Sourav Das, and Alexander Spiegelman. 2025. Shoal++: High Throughput DAG BFT Can Be Fast and Robust!. In *Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association.
- [4] Balaji Arun and Binoy Ravindran. 2022. Scalable Byzantine Fault Tolerance via Partial Decentralization. *Proc. of the VLDB Endowment* 15, 9 (2022), 1739–1752.
- [5] Kushal Babel, Andrey Chursin, George Danezis, Anastasios Kichidis, Lefteris Kokoris-Kogias, Arun Koshy, Alberto Sonnino, and Mingwei Tian. 2025. MYSTICETI: Reaching the Latency Limits with Uncertified DAGs. In *Network and Distributed Systems Security Symposium (NDSS)*.
- [6] Johannes Behl, Tobias Distler, and Rüdiger Kapitza. 2015. Consensus-oriented parallelization: How to earn your first million. In *Annual Middleware Conf. (Middleware)*. 173–184.
- [7] Johannes Behl, Tobias Distler, and Rüdiger Kapitza. 2017. Hybrids on steroids: SGX-based high performance BFT. In *European Conf. on Computer Systems (EuroSys)*. 222–237.
- [8] Yehonatan Buchnik and Roy Friedman. 2020. FireLedger: a high throughput blockchain consensus protocol. *Proceedings of the VLDB Endowment* 13, 9 (2020), 1525–1539.
- [9] Miguel Castro and Barbara Liskov. 1999. Practical Byzantine fault tolerance. In *Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 173–186.
- [10] Benjamin Y Chan and Elaine Shi. 2020. Streamlet: Textbook streamlined blockchains. In *Conf. on Advances in Financial Technologies (AFT)*. ACM, 1–11.
- [11] TH Hubert Chan, Rafael Pass, and Elaine Shi. 2018. Pala: A simple partially synchronous blockchain. *Cryptology ePrint Archive* (2018).
- [12] TH Hubert Chan, Rafael Pass, and Elaine Shi. 2018. PiLi: An Extremely Simple Synchronous Blockchain. *Cryptology ePrint Archive* (2018).
- [13] Feng Cheng, Jiang Xiao, Cunyang Liu, Shijie Zhang, Yifan Zhou, Bo Li, Baochun Li, and Hai Jin. 2024. Shardag: Scaling dag-based blockchains via adaptive sharding. In *Int. Conf. on Data Engineering (ICDE)*. IEEE, 2068–2081.
- [14] Allen Clement, Edmund L Wong, Lorenzo Alvisi, Michael Dahlin, and Mirco Marchetti. 2009. Making Byzantine Fault Tolerant Systems Tolerate Byzantine Faults. In *Symposium on Networked Systems Design and Implementation (NSDI)*, Vol. 9. USENIX Association, 153–168.
- [15] Carlo Curino, Evan Philip Charles Jones, Yang Zhang, and Samuel R Madden. 2010. Schism: a workload-driven approach to database replication and partitioning. *Proceedings of the VLDB Endowment* 3, 1 (2010).
- [16] Xiaohai Dai, Wei Li, Guanxiong Wang, Jiang Xiao, Haoyang Chen, Shufei Li, Albert Y Zomaya, and Hai Jin. 2024. Remora: A low-latency dag-based bft through optimistic paths. *Transactions on Computers* (2024).
- [17] Xiaohai Dai, Guanxiong Wang, Jiang Xiao, Zhengxuan Guo, Rui Hao, Xia Xie, and Hai Jin. 2024. LightDAG: A low-latency DAG-based BFT consensus through lightweight broadcast. In *Int. Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 998–1008.
- [18] Xiaohai Dai, Zhaonan Zhang, Zhengxuan Guo, Chaozheng Ding, Jiang Xiao, Xia Xie, Rui Hao, and Hai Jin. 2024. Wahoo: A dag-based bft consensus with low

- latency and low communication overhead. *Transactions on Information Forensics and Security* (2024).
- [19] Xiaohai Dai, Zhaonan Zhang, Jiang Xiao, Jingtao Yue, Xia Xie, and Hai Jin. 2023. GradedDAG: An asynchronous DAG-based BFT consensus with lower latency. In *Int. Symposium on Reliable Distributed Systems (SRDS)*. IEEE, 107–117.
 - [20] George Danezis, Lefteris Kokoris-Kogias, Alberto Sonnino, and Alexander Spiegelman. 2022. Narwhal and Tusk: a DAG-based mempool and efficient BFT consensus. In *European Conf. on Computer Systems (EuroSys)*. 34–50.
 - [21] Hung Dang, Tien Tuan Anh Dinh, Dumitrel Loghin, Ee-Chien Chang, Qian Lin, and Beng Chin Ooi. 2019. Towards Scaling Blockchain Systems via Sharding. In *Proceedings of the 2019 International Conference on Management of Data (Amsterdam, Netherlands) (SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 123–140. <https://doi.org/10.1145/3299869.3319889>
 - [22] Dmitry Duplyakin, Robert Ricci, Aleksander Maricq, Gary Wong, Jonathon Duerig, Eric Eide, Leigh Stoller, Mike Hibler, David Johnson, Kirk Webb, et al. 2019. The Design and Operation of {CloudLab}. In *Annual Technical Conf. (ATC)*. USENIX Association, 1–14.
 - [23] Jose M Faleiro and Daniel J Abadi. 2015. Rethinking serializable multiversion concurrency control. *Proceedings of the VLDB Endowment* 8, 11 (2015), 1190–1201.
 - [24] Jose M Faleiro, Daniel J Abadi, and Joseph M Hellerstein. 2017. High performance transactions via early write visibility. *Proc. of the VLDB Endowment* 10, 5 (2017), 613–624.
 - [25] Fangyu Gai, Jianyu Niu, Ivan Beschastnikh, Chen Feng, and Sheng Wang. 2023. Scaling blockchain consensus via a robust shared mempool. In *Int. Conf. on Data Engineering (ICDE)*. IEEE, 530–543.
 - [26] Yossi Gilad, Rotem Hemo, Silvio Micali, Georgios Vlachos, and Nikolai Zeldovich. 2017. Algorand: Scaling byzantine agreements for cryptocurrencies. In *Symposium on Operating Systems Principles (SOSP)*. ACM, 51–68.
 - [27] Neil Girdharan, Florian Suri-Payer, Ittai Abraham, Lorenzo Alvisi, and Natacha Crooks. 2024. Autobahn: Seamless high speed BFT. In *Symposium on Operating Systems Principles (SOSP)*. ACM SIGOPS, 1–23.
 - [28] Guy Golan Gueta, Ittai Abraham, Shelly Grossman, Dahlia Malkhi, Benny Pinkas, Michael K Reiter, Dragos-Adrian Seredinschi, Orr Tamir, and Alin Tomescu. 2019. SBFT: a Scalable Decentralized Trust Infrastructure for Blockchains. In *Int. Conf. on Dependable Systems and Networks (DSN)*. IEEE/IFIP, 568–580.
 - [29] Suyash Gupta, Jelle Hellings, and Mohammad Sadoghi. 2021. Rcc: Resilient concurrent consensus for high-throughput secure transaction processing. In *Int. Conf. on Data Engineering (ICDE)*. IEEE, 1392–1403.
 - [30] Timo Hanke, Mahnush Movahedi, and Dominic Williams. 2018. Dfinity technology overview series, consensus system. *arXiv preprint arXiv:1805.04548* (2018).
 - [31] Huawei Huang, Xiaowen Peng, Jianzhou Zhan, Shenyang Zhang, Yue Lin, Zibin Zheng, and Song Guo. 2022. Brokerchain: A cross-shard blockchain protocol for account/balance-based state sharding. In *IEEE INFOCOM 2022-IEEE Conference on Computer Communications*. IEEE, 1968–1977.
 - [32] Philipp Jovanovic, Lefteris Kokoris-Kogias, Bryan Kumara, Alberto Sonnino, Pasindu Tennage, and Igor Zablotchi. 2024. Mahi-mahi: Low-latency asynchronous bft dag-based consensus. *arXiv preprint arXiv:2410.08670* (2024).
 - [33] Idit Keidar, Eleftherios Kokoris-Kogias, Oded Naor, and Alexander Spiegelman. 2021. All you need is dag. In *Symposium on Principles of Distributed Computing (PODC)*. ACM, 165–175.
 - [34] Mahimna Kelkar, Soubhik Deb, Sishan Long, Ari Juels, and Sreeram Kannan. 2023. Themis: Fast, strong order-fairness in byzantine consensus. In *SIGSAC Conf. on Computer and Communications Security (CCS)*. ACM, 475–489.
 - [35] Mahimna Kelkar, Fan Zhang, Steven Goldfeder, and Ari Juels. 2020. Order-fairness for byzantine consensus. In *Annual Int. Cryptology Conf.* Springer, 451–480.
 - [36] Leonard Kleinrock. 1974. *Queueing systems: theory*. Vol. 2. wiley.
 - [37] Quentin Knipf, Lefteris Kokoris-Kogias, Alberto Sonnino, Igor Zablotchi, and Nuda Zhang. 2025. Pilotfish: Distributed Execution for Scalable Blockchains. In *Int. Conf. on Financial Cryptography and Data Security (FC)*. Springer, 287–306.
 - [38] Eleftherios Kokoris-Kogias. [n.d.]. Robust and Scalable Consensus for Sharded Distributed Ledgers. ([n.d.]).
 - [39] Ramakrishna Kotla, Lorenzo Alvisi, Mike Dahlin, Allen Clement, and Edmund Wong. 2007. Zyzzyva: speculative byzantine fault tolerance. *Operating Systems Review (OSR)* 41, 6 (2007), 45–58.
 - [40] Jae Kwon. 2014. Tendermint: Consensus without mining. (2014).
 - [41] Qian Lin, Pengfei Chang, Gang Chen, Beng Chin Ooi, Kian-Lee Tan, and Zhengkui Wang. 2016. Towards a non-2pc transaction management in distributed database systems. In *Int. Conf. on Management of Data (SIGMOD)*. 1659–1674.
 - [42] Yu-Shan Lin, Ching Tsai, Tz-Yu Lin, Yun-Sheng Chang, and Shan-Hung Wu. 2021. Don't look back, look into the future: Prescient data partitioning and migration for deterministic database systems. In *Proceedings of the 2021 International Conference on Management of Data*. 1156–1168.
 - [43] Yi Lu, Xiangyao Yu, Lei Cao, and Samuel Madden. 2020. Aria: a fast and practical deterministic OLTP database. *Proc. VLDB Endow.* 13, 12 (July 2020), 2047–2060. <https://doi.org/10.14778/3407790.3407808>
 - [44] Hanzheng Lyu, Shaokang Xie, Jianyu Niu, Ivan Beschastnikh, Yinqian Zhang, Mohammad Sadeghi, and Chen Feng. 2025. Orthrus: accelerating multi-BFT consensus through concurrent partial ordering of transactions. In *Int. Conf. on Data Engineering (ICDE)*. IEEE, 2615–2627.
 - [45] Hanzheng Lyu, Shaokang Xie, Jianyu Niu, Chen Feng, Yinqian Zhang, and Ivan Beschastnikh. 2025. Ladon: High-performance multi-BFT consensus via dynamic global ordering. In *European Conf. on Computer Systems (EuroSys)*. 226–242.
 - [46] Dahlia Malkhi, Chrysoula Stathakopoulou, and Maofan Yin. 2024. BBKA-CHAIN: Low latency, high throughput BFT consensus on a DAG. In *Int. Conf. on Financial Cryptography and Data Security (FC)*. Springer, 51–73.
 - [47] Bhavana Mehta, Nupur Baghel, Mohammad Javad Amiri, Boon Thau Loo, and Ryan Marcus. 2025. Adaptive Sharding in Untrusted Environments. *Proceedings of the ACM on Management of Data (SIGMOD)* 3, 6 (2025), 1–28.
 - [48] Heena Nagda, Sidharth Sankhe, Sakshi Sinha, Keon Attarha, Mohammad Javad Amiri, and Boon Thau Loo. 2026. DAG of DAGs: Order-Fairness Made Practical. In *SIGMOD Int. Conf. on Management of Data*. ACM.
 - [49] Abdul Qamar, K. Ashwin Kumar, and Amol Deshpande. 2013. SWORD: Scalable Workload-Aware Data Placement for Transactional Workloads. In *Proceedings of the 16th International Conference on Extending Database Technology (EDBT)*. 430–441. <https://doi.org/10.1145/2452376.2452427>
 - [50] Mayank Raikwar, Nikita Polyanski, and Sebastian Müller. 2024. SoK: DAG-Based Consensus Protocols. In *Int. Conf. on Blockchain and Cryptocurrency (ICBC)*. IEEE, 1–18.
 - [51] Marco Serafini, Rebecca Taft, Aaron J. Elmore, Andrew Pavlo, Ashraf Aboulmaga, and Michael Stonebraker. 2016. Clay: Fine-Grained Adaptive Partitioning for General Database Schemas. *Proceedings of the VLDB Endowment* 10, 4 (2016), 445–456. <https://doi.org/10.14778/3025111.3025125>
 - [52] Nibesh Shrestha, Rohan Shrothrium, Aniket Kate, and Kartik Nayak. 2024. Sailfish: Towards Improving the Latency of DAG-based BFT. In *Symposium on Security and Privacy (SP)*. IEEE, 21–21.
 - [53] Alexander Spiegelman, Balaji Arun, Rati Gelashvili, and Zekun Li. 2024. Shoal: Improving dag-bft latency and robustness. In *Int. Conf. on Financial Cryptography and Data Security (FC)*. Springer, 92–109.
 - [54] Alexander Spiegelman, Neil Girdharan, Alberto Sonnino, and Lefteris Kokoris-Kogias. 2022. Bullshark: Dag bft protocols made practical. In *ACM SIGSAC Conf. on Computer and Communications Security (CCS)*. 2705–2718.
 - [55] Chrysoula Stathakopoulou, Tudor David, Matej Pavlovic, and Marko Vukolic. 2022. Mir-bft: Scalable and robust bft for decentralized networks. *Journal of Systems Research* 2, 1 (2022), 3.
 - [56] Suiscan. 2026. Suiscan Project Directory. <https://suiscan.xyz/mainnet/apps/directory/>. Accessed 2026.
 - [57] Rebecca Taft, Essam Mansour, Marco Serafini, Jennie Duggan, Aaron J. Elmore, Ashraf Aboulmaga, Andrew Pavlo, and Michael Stonebraker. 2014. E-Store: Fine-Grained Elastic Partitioning for Distributed Transaction Processing Systems. *Proceedings of the VLDB Endowment* 8, 3 (2014), 245–256. <https://doi.org/10.14778/2735508.2735514>
 - [58] Alexander Thomson, Thaddeus Diamond, Shu-Chun Weng, Kun Ren, Philip Shao, and Daniel J Abadi. 2012. Calvin: fast distributed transactions for partitioned database systems. In *SIGMOD Int. Conf. on Management of Data*. ACM, 1–12.
 - [59] Giorgos Tsimos, Anastasios Kichidis, Alberto Sonnino, and Lefteris Kokoris-Kogias. 2024. Hammerhead: Leader reputation for dynamic scheduling. In *2024 IEEE 44th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 1377–1387.
 - [60] Giuliana Santos Veronese, Miguel Correia, Alysson Neves Bessani, and Lau Cheuk Lung. 2009. Spin one's wheels? Byzantine fault tolerance with a spinning primary. In *Int. Symposium on Reliable Distributed Systems (SRDS)*. IEEE, 135–144.
 - [61] Giuliana Santos Veronese, Miguel Correia, Alysson Neves Bessani, and Lau Cheuk Lung. 2010. EBAA: Efficient Byzantine agreement for wide-area networks. In *Int. Symposium on High Assurance Systems Engineering (HASE)*. IEEE, 10–19.
 - [62] Maofan Yin, Dahlia Malkhi, Michael K Reiter, Guy Golan Gueta, and Ittai Abraham. 2019. HotStuff: BFT consensus with linearity and responsiveness. In *Symposium on Principles of Distributed Computing (PODC)*. ACM, 347–356.
 - [63] Yunhao Zhang, Srinath Setty, Qi Chen, Lidong Zhou, and Lorenzo Alvisi. 2020. Byzantine ordered consensus without Byzantine oligarchy. In *Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 633–649.